

Empowering the Graph Thinkers in the Age of Generative AI

By Dinis Cruz and ChatGPT Deep Research – June 2025

Introduction

In a world increasingly defined by complex interconnections, a unique set of individuals thrives on seeing relationships and patterns everywhere – the **"graph thinkers."** These are people who intuitively view information as networks of nodes and links, believing that *"everything is really just graphs and maps"* ¹. Graph thinkers can mentally connect disparate concepts across business functions, technology stacks, or even social systems into coherent knowledge graphs. Historically, however, many of these creative minds have been constrained by the **limitations of technology and coding skills**. Realizing a vision that "everything connects to everything" often required heavy technical implementation, dedicated development teams, or costly graph database infrastructure – resources that were out of reach for most individual innovators.

Today, recent advances in **Generative AI (GenAI)** and **no-code development** tools promise to change this landscape. Large language models (LLMs) and AI-assisted development platforms are democratizing the ability to build sophisticated applications without traditional programming ². In particular, GenAI is enabling a new paradigm: using LLMs as *ephemeral graph databases* to create and manipulate knowledge graphs on the fly. This white paper explores how individuals who "think in graphs" can leverage generative AI and modern graph technologies to finally implement their visions at the **speed of thought**, without being blocked by the historical barriers of coding or rigid software tools.

We will discuss the challenges graph thinkers faced in the past, the emerging opportunities with LLMs as graph engines, and the open-source innovations (such as **MGraph-DB** and **serverless semantic knowledge graphs**) that are paving the way. In doing so, we aim to provide a roadmap for graph thinkers – and the organizations that stand to benefit from their insights – to harness GenAI in building dynamic, living knowledge structures. The result is a democratization of innovation: anyone with a graph-oriented mindset can now turn ideas into tangible models and applications, fostering creativity and problem-solving in areas from business process mapping to enterprise knowledge management.

The Challenge for Graph Thinkers

Graph thinkers are a rare but valuable breed of innovators who naturally view problems through the lens of relationships and networks. Rather than thinking in isolated silos or linear processes, they see webs of interdependence: in their eyes, *"everything is a graph"* connecting to everything else. This cognitive style can be applied to countless domains – from linking components of a business strategy, to mapping software system architecture, to understanding cause-and-effect in social or security contexts. Such individuals excel at envisioning **holistic models** of complex systems. For example, a graph thinker in cybersecurity might connect vulnerabilities to systems, systems to business impacts, and threats to mitigations in one mental map, revealing insights that a linear list would miss.

However, turning these rich mental graphs into reality has historically been **frustrating and difficult** for those without strong programming abilities. In the past, an individual might sketch an intricate concept map on a whiteboard, only to hit a wall when trying to implement it in a software tool or database. Traditional development processes were too slow and rigid to keep up with the exploratory, evolutionary nature of graph thinking. When a graph-oriented person handed off their ideas to a software development team, the typical question from the developers – “tell us exactly what you want built” – was at odds with the graph thinker’s *emergent* and iterative approach. Before GenAI, unless one was both a graph thinker *and* a skilled coder, many brilliant ideas remained stuck at the conceptual stage.

Several factors made this so:

- **Dependency on Technical Teams:** Graph thinkers who lacked coding skills had to rely on engineers to translate their vision into software. This introduced delays, communication gaps, and often a loss of fidelity in the ideas. Only a lucky few who worked in well-resourced teams or had the funds to hire developers could see their graph concepts fully realized; countless others were left without a path to implementation.
- **Imperfect Tools:** The software available for modeling graphs (such as conventional mind-mapping tools or enterprise modeling software) often fell short. They could not capture the dynamic, multi-dimensional nature of how graph thinkers see the world. Either the tools were too simplistic, forcing rich graphs into tree hierarchies, or they were too complex and technical (like traditional graph databases) requiring significant setup and maintenance.
- **Slow Iteration Cycles:** Perhaps most critically, the speed of traditional development was incompatible with the speed of thought. Graph thinking is an exploratory process – you often don’t know in advance what the “final” graph should look like. You need to add nodes and connections, see the result, refine the structure, and repeat. Waiting weeks for a development team to produce a new feature or adjust a data model stifles this creative process. The lack of *real-time feedback* meant that by the time a prototype was built, the graph thinker’s ideas might have already evolved in a different direction.

These challenges were exacerbated by the state of graph technology itself. **Graph databases** – specialized databases for storing and querying network-structured data – have been around for decades, but they did not evolve in a way that served individual innovators working on small, rapidly changing graphs. Many graph databases (e.g., Neo4j, Amazon Neptune) were designed to handle massive, enterprise-scale knowledge graphs with fixed schemas. They excel at storing billions of relationships and answering complex queries, but they are **heavyweight and inflexible** for an individual who wants to spin up a fresh, experimental graph model overnight. Deploying and maintaining these systems required significant infrastructure and expertise, which was impractical for one-off personal projects or fast brainstorming.

Moreover, traditional graph databases assume you have a well-defined schema or ontology upfront – a structure of node and edge types that remains relatively stable. Graph thinkers, on the other hand, often deal with **dynamic ontologies** that evolve as understanding grows. Imposing a rigid schema too early would be like setting stone before the idea has taken shape. In practice, this meant graph thinkers either had to shoehorn their ideas into existing tools (losing nuance), or give up on using graph databases entirely for lack of agility.

In summary, until recently, the immense potential of graph thinkers was largely **bottlenecked by technology**. The people who could most benefit from advanced graph modeling were the least empowered to use it. This is where the convergence of **no-code development platforms** and **Generative AI** begins to change the game.

The Rise of GenAI and No-Code Solutions

The past few years have witnessed a revolution in how software is built and who can build it. **No-code and low-code platforms** have lowered the barrier for creating applications by offering visual interfaces, drag-and-drop components, and templates that non-programmers can use. This trend has already started democratizing app development, enabling “citizen developers” to turn ideas into working tools without writing code ³ ⁴. But the advent of **Generative AI**, especially large language models like GPT-4, has supercharged this democratization.

Generative AI (GenAI) models can produce code, text, and structured outputs from plain language prompts. They act as copilots or assistants that understand a user’s intent and generate the necessary logic or content. For graph thinkers, this means an **AI translator** now exists between their conceptual ideas and the technical implementation. With GenAI, even those with *zero programming knowledge* can describe what they want in natural language and have the AI produce working code or configurations. As one observer noted, “*genAI removes the entry barriers to coding development*”, allowing regular users to create software without needing to manually write any code ².

Key developments that empower graph thinkers include:

- **AI-Assisted Coding:** Tools like OpenAI’s Codex, GitHub Copilot, and various chat-based coding assistants can generate code for a given task description. A graph thinker can say “create a data structure to represent connections between these entities” and the AI will provide code (in Python, JavaScript, etc.) to create that graph structure. This vastly speeds up prototyping. The AI can also generate *visualization code* (e.g. Graphviz diagrams, D3.js networks) from a description of how the user wants to see the graph.
- **Natural Language Querying:** Modern LLMs can interpret natural language questions about data. This means a graph thinker could load data and then ask, “how are A and B connected?” or “find clusters of related concepts,” and the AI will parse the graph and respond with the answer or even produce a subgraph highlighting the connection. What used to require writing a complex query in graph query languages (like Cypher or Gremlin) can now be done with plain English questions.
- **Structured Outputs from LLMs:** Critically, LLMs today can output not just free-flowing text but structured data such as JSON, XML, or Python objects that represent complex data structures. By providing a schema or examples, users can have the AI return a list of nodes and edges extracted from text, for instance. Dinis Cruz demonstrated this by prompting LLMs to output structured JSON of entities and relationships found in RSS news articles ⁵ ⁶. This technique turns unstructured information into a **machine-readable graph representation** via AI.
- **Rapid Iteration via Conversation:** Using an AI in a chat loop allows graph thinkers to refine their model interactively. They can generate an initial graph, examine it, then ask the AI to adjust the structure, add missing connections, or apply a different ontology. This conversational, iterative development was nearly impossible in the old model of filing tickets to a dev team and waiting weeks. Now it can happen in minutes, following the user’s train of thought.

In essence, **GenAI acts as the ultimate no-code platform** for graph-based development. It brings the speed, flexibility, and creativity that graph thinkers need. Ideas can be tested and visualized almost as quickly as they are conceived. Furthermore, AI assistance helps overcome the *knowledge gap* – the graph thinker doesn't need to know the intricacies of graph database query languages, server setup, or programming syntax. They only need to focus on the relationships and logic of the problem, and let the AI handle the execution.

The impact of this is profound: **imagination is no longer limited by implementation**. A whole class of innovators can now actively build, not just brainstorm. As generative AI matures, we foresee an explosion of custom micro-applications, knowledge models, and decision-support tools built by domain experts who think in graphs – people who previously could not bring their ideas to life due to technical barriers. Studies already indicate that integrating GenAI with low-code platforms leads to more sophisticated and efficient development by non-coders ⁷ ⁸ .

However, unlocking this potential requires more than just the AI. There must be an underlying framework to store, manipulate, and visualize the graphs that the AI helps to create. This is where new approaches in graph database architecture come into play, especially the concept of **using LLMs themselves as ephemeral graph databases**.

LLMs as Ephemeral Graph Databases

One of the most intriguing emerging patterns is to use a **large language model as a graph database** – not in the traditional persistent sense, but as an *ephemeral, on-demand* knowledge store. The idea is unconventional: normally, we think of a database as something static and persistent, where data is stored and remains until queried. But with powerful LLMs, we can instead feed the model data and a query in one go, and have it perform complex graph-like reasoning internally, returning the results without ever explicitly storing the full data on disk in a structured database.

In practical terms, treating an LLM as an ephemeral graph database works like this:

- 1. Prepare Data as Text/JSON:** Start with your raw data – it could be a collection of documents, a CSV of transactions, or a set of statements about a domain. The first step is to convert this data into a *prompt-friendly representation*. For graph use cases, this often means formatting the data as nodes and edges in text or JSON. For example, one might represent a list of relationships like: `NodeA -> NodeB (relationType)`, or use JSON structures listing entities and their connections. Dinis Cruz's MyFeeds project, for instance, converted RSS feed articles into a JSON object and then into a tree-structured text that represents the graph of entities ⁹ ¹⁰ .
- 2. Load into the LLM via Prompt:** Provide the LLM with a prompt that includes both the data (or relevant subset of it) and an instruction (the "query"). Essentially, you are telling the LLM, "Here is a graph (described in text), now answer this question about it or transform it in some way." Because the LLM has been trained on vast amounts of text (including likely graph-like data structures and reasoning patterns), it can parse the input as a graph and perform reasoning. This could be querying a relationship path, finding clusters, or creating a new node by merging others – all tasks that a graph database could do.
- 3. LLM Processes and Transforms:** The LLM, within a single prompt-response cycle, acts as the engine that *loads* the graph (in its neural network working memory), carries out the instructions, and formulates an answer. Importantly, this answer can be structured. For example, you might ask the LLM to *"find all connections between X and Y within two hops and return them as JSON"*

edges". With features like OpenAI's function calling or structured output schemas, the LLM can output a machine-readable result – effectively the answer subgraph ⁵ ⁶ .

4. **Output to Files:** The result from the LLM can be saved back to the file system (e.g., as a JSON or CSV file). This could represent a filtered set of relationships, a summary, or any transformed data. Because the output is now on disk, it becomes the input for the next round of processing. In a sense, the **file system plus LLM together form a feedback loop**: data files → LLM prompt → output files → (back to LLM or user).
5. **Iterate as Needed:** The next prompt can take the newly created file (which might itself describe a graph or partial graph) and do further operations. For example, one round might extract an ontology (types of entities and relations), the next round might use that ontology to categorize or filter the graph, and a subsequent round could generate a visualization code. Each step the "graph database" is instantiated fresh in the LLM's memory from the files, manipulated, and then dissolved after output. Hence, it's *ephemeral* – existing only for the duration of each prompt – yet across prompts a persistent result evolves, stored in the series of files.

This approach is powerful for several reasons:

- **No Permanent Infrastructure:** You don't need to set up a Neo4j server or similar. The heavy lifting is done by the LLM in the context of a prompt. The "database" disappears when the prompt is done, so there's zero cost when you're not querying. This aligns perfectly with *serverless computing* principles – you pay (or use compute) only when a query is executed, and there's no always-on server to maintain.
- **Dynamic Schema:** The structure of the graph can be fluid. In one prompt, you might treat the data as one schema (say a property graph with attributes on nodes), and in the next, you could reshape it (turning some attributes into first-class nodes or vice versa). Traditional graph DBs struggle with changing schema or ontology on the fly; an LLM doesn't, because it just follows the prompt's description each time. You can effectively have **different views of the same data as different graphs** without data migration – simply by changing how you prompt the LLM to interpret the data.
- **Combining Transformation with Query:** In databases, typically you must load data, then query it. Here, the load-transform-query can happen in one step. For instance, if the source data is not in a graph form, the prompt's first part can instruct the LLM to *interpret or transform* it into a graph structure, and the second part of the prompt is the query on that structure. The user of course doesn't see these as separate – they just see the final answer. Yet conceptually, the LLM did the ETL (extract-transform-load) and the query *all at once*. This ephemeral graph creation means we don't need a long pre-processing pipeline; we trade storage for computation.
- **Ephemeral = Secure and Fresh:** Because the graph in the LLM is transient, there's less risk of stale data. Each query can incorporate the most up-to-date files. It also means sensitive data isn't stored long-term in a database – it lives in memory for a moment and then is gone (assuming we don't log prompts). Of course, one must still trust the LLM provider with the data during that moment, so this approach might be best paired with self-hosted or open-source LLMs for very sensitive applications.

Real-world early examples of this paradigm are emerging. Microsoft Research's **GraphRAG** technique is one notable instance where LLMs generate a knowledge graph on the fly for each query to improve

question-answering on private data ¹¹ ¹² . In GraphRAG, the LLM reads a set of documents and produces a temporary graph of entities and relationships relevant to the question; that graph is then used to fetch answers more effectively than traditional retrieval methods. This underscores that LLMs are capable of performing graph construction and traversal as part of their native functionality. The graph is essentially an *ephemeral byproduct* of the prompt, not a persisted structure – yet it adds significant value in accuracy and explainability of the AI’s answer.

In summary, using LLMs as ephemeral graph databases opens up a new **flexible workflow** for graph thinkers:

- They can **feed raw knowledge** (files, text dumps, etc.) into an LLM and ask it to construct connections.
- They can **query relationships** in natural language, and get back structured answers or even new graphs.
- They can do this iteratively, **refining the graph** with each step, without ever dealing with a schema migration or a graph query language syntax.
- Every intermediate result can be captured as a file (like a JSON graph or a visualization image), creating a trail of outputs that document the graph’s evolution – useful for provenance and explainability.

This workflow was practically unheard of a few years ago. It’s a direct result of LLMs reaching a level of sophistication where they can internalize and manipulate data structures based on prompts. It shifts the role of the human to **curator and conductor** of knowledge, rather than coder or database administrator. The graph thinker describes what they want, and the LLM does the rest, one ephemeral graph at a time.

Memory-First Graph Databases and Serverless Architecture

While LLMs themselves can act as ephemeral data stores, there is also a complementary development in the world of graph databases that perfectly aligns with the needs of graph thinkers: **memory-first, serverless graph databases**. One such open-source innovation, developed by Dinis Cruz and collaborators, is **MGraph-DB (also referred to as MGraph-AI)** – a graph database designed from the ground up for GenAI applications and serverless environments ¹³ .

MGraph-DB takes a fundamentally different approach from traditional graph databases:

- It is **memory-first**, meaning it keeps the entire graph in memory during operation for speed, and only writes to storage when needed for persistence ¹⁴ ¹⁵ . This makes graph operations (traversals, additions, queries) extremely fast, which is important when an AI or interactive user is querying the graph in real-time.
- It uses **JSON serialization to persist data** to the file system (or cloud storage) as needed ¹⁴ . Essentially, the graph is stored as JSON files when at rest. This aligns well with how LLMs produce output (JSON) and how serverless functions handle state (often reading/writing to cloud storage). JSON is human-readable and easy to version control, diff, and manage, which gives graph thinkers transparency into their data.
- **Serverless and Cloud-native:** MGraph-DB was built to run in serverless contexts with zero cost when not in use ¹⁶ ¹⁷ . Traditional graph databases often run as always-on services; in contrast, a memory-first DB can spin up in a lambda function or cloud run instance, load JSON

data, answer queries, and shut down – incurring cost only for the milliseconds it was active. This is ideal for use cases where graph operations are intermittent and event-driven, such as on-demand analysis or triggered workflows.

- **Use of Cloud Storage as the Database:** Rather than a proprietary store, MGraph-DB treats something like Amazon S3 as the backing store for data ¹⁷. Graph data (in JSON files) can be saved to S3 in a structured way. For example, one can store each node or subgraph as a separate file, or snapshot entire graph states as files. Because S3 (or any cloud object storage) is cheap, scalable, and inherently versioned via keys, it becomes a simple but effective graph repository. In practice, this means **using S3 like a database**, with the benefit that any tool or process that can read JSON from S3 can integrate with the graph data – there’s no special query interface needed beyond basic file I/O.
- **Type-Safety and Schema as Code:** MGraph-DB provides a *clean API with type-safe classes* to define nodes, edges, and attributes ¹⁸. This means graph thinkers (with minimal Python knowledge or via AI assistance) can define their domain model in code, ensuring that, for instance, a "Person" node has a name property, or an "Order" node links to a "Customer" node. The type-safe layer catches errors early (e.g., trying to link incompatible node types) and makes the graph more robust. Yet, because everything is still stored as JSON, the schema can be evolved with code changes and migration of JSON – simpler than dealing with a rigid database schema migration.
- **Minimal Dependencies:** By keeping the system lightweight (it’s essentially a Python library with JSON read/write), MGraph-DB avoids heavy dependencies. This makes it easy to embed in AI workflows and deploy on various platforms, from cloud functions to edge devices. **Graph operations** like searching connections, filtering nodes, or editing relationships are all provided as methods (e.g., `MGraph__Filter`, `MGraph__Edit` classes) ¹⁹ that operate on the in-memory graph. These can be invoked by AI-generated code or directly by users.

In practice, tools like MGraph-DB empower graph thinkers to **set up their own graph sandbox quickly**. For example, a user could instantiate an MGraph-DB in a notebook or script, feed it JSON data (perhaps extracted by an LLM), perform some in-memory graph analysis or merging, and then dump the results back to JSON. This has been used in projects like MyFeeds.ai, where after extracting entities and relationships via LLMs, the data is merged into an MGraph-DB to facilitate easy combination and manipulation of those JSON objects as actual graph nodes and edges ²⁰ ²¹. In that workflow, MGraph-DB acted as the *glue* that holds intermediate graph data, allowing multiple LLM calls to contribute pieces that are combined into a single knowledge graph.

An illustrative example of this architecture in action is the **“serverless semantic knowledge graph” pipeline** used for building a news feed knowledge graph ²². In this pipeline:

- A cloud function (FastAPI on AWS Lambda) triggers periodically (e.g., hourly) ²³.
- It fetches an RSS feed (news articles) and stores the raw feed XML to S3 ²³.
- It converts the XML to JSON and saves that to S3.
- It then uses MGraph-DB to create a timeline graph of article publication dates (each article becomes a node linked to its date, month, year) – this graph is stored as a `.mgraph.json` file on S3 ²⁴ ²⁵.
- The graph is also exported to a DOT format (Graphviz) text and rendered via another Lambda into a PNG image ²⁶.

- Both the data and visualization are saved to S3, with two versions kept: a timestamped version for historical provenance and a “latest” version for easy access ²⁷. For instance, the S3 folder will have `.../latest/feed-timeline.mgraph.json` and also `.../2025/02/20/16/feed-timeline.mgraph.json` for the specific run at 4 PM on Feb 20, 2025 ²⁷. This scheme provides an audit trail of how the graph changes over time (crucial for explainability and deterministic behavior) while also giving a stable path for consumers to fetch the most recent graph.

This example shows how using cloud storage as a database, combined with ephemeral compute, achieves what a traditional always-on graph database would do – but in a more **cost-effective and flexible** manner. Each piece of data in the pipeline is an artifact on the file system (S3): raw data, intermediate JSON, final graph, visualization image. A graph thinker could inspect or even manually edit those if needed, or rerun parts of the pipeline easily by invoking the functions.

For open-source graph innovation, **MGraph-DB** and similar projects represent an important shift. They align with the way graph thinkers operate: load everything in memory (as one would mentally), see it all at once, manipulate freely, then store snapshots. The historical approach of tuning a database for either transactions or giant analytics is replaced by an approach optimized for **AI workloads and knowledge exploration** ¹⁷ ²⁸. Features like version control of graphs (since diffs can be done on JSON files) and integration with semantic web standards are built-in ²⁹ ³⁰, acknowledging that graphs often need to represent richly typed relationships (ontologies, taxonomies) and that those representations may change and improve with feedback.

In essence, tools like MGraph-DB provide the **scaffolding** on which LLM-driven workflows can run reliably. An LLM might generate or update a set of triplets (subject-predicate-object statements) which are then inserted into the MGraph in memory; another LLM call might ask a question whose answer requires traversing the combined graph – that could be handled either by the LLM reasoning or by a query to the in-memory graph via code. The result can be fed back into the next AI prompt or visualized for the user.

The synergy between *ephemeral LLM graphs* and *memory-first graph databases* is powerful. The LLM can handle the fuzzier side – extracting knowledge, making connections with guidance in natural language – and the lightweight graph DB handles the deterministic side – ensuring data integrity, providing fast lookups, and storing results. Both are highly complementary in a serverless paradigm: spin them up when needed, tear them down after use, and use simple storage (files) to link stages together.

To summarize this section, the technologies now at the disposal of graph thinkers include:

- **LLMs as Graph Engines:** Able to interpret and generate graph-structured data on the fly, great for one-off reasoning and transformation tasks.
- **Memory-First Graph DBs (e.g., MGraph-DB):** Fast, in-memory manipulation of graphs with persistence to simple storage, ideal for integrating multiple AI calls and keeping a consistent working state.
- **Cloud Storage as a Graph Repository:** Cheap, scalable storage where graph data can be dumped and versioned, eliminating the need for complex database maintenance.
- **Visualization Tools:** With Graphviz, D3, or other libraries (often also invoked via AI-generated code), graph thinkers can automatically produce visual maps of their knowledge graphs to better understand them. Dinis Cruz emphasizes that visualization is key, as he “*can’t really understand and visualise what the graphs actually look like*” without it ³¹.

We now have an unprecedented **toolkit** for turning mental graphs into living graphs. Next, we look at how these capabilities can be applied to real-world scenarios and the impact that empowering graph thinkers can have in various domains.

Applications: From Business Processes to Personal Knowledge Graphs

The convergence of graph thinking and GenAI-based tools opens up a multitude of impactful applications. Here we focus on a few key areas where graph thinkers, equipped with LLMs and serverless graph tech, can deliver unique value:

1. Mapping Business Processes and Organisational Knowledge

One of the greatest opportunities lies in using graph approaches to model the complex processes and knowledge within organizations. Every company has vast amounts of *tacit knowledge* – how things really get done, how data flows between departments, who depends on whom for decisions, what the unofficial workarounds are, and so on. Traditionally, capturing this in a usable form has been incredibly hard. It often ends up as static flowcharts, documents, or outdated intranet pages that fail to reflect the real, living system.

Graph thinkers can change this by building **semantic knowledge graphs** of the enterprise. Using GenAI, they can:

- **Interview and Extract Knowledge:** By using LLMs to analyze meeting transcripts, emails, or by interactively querying subject matter experts, they can extract entities (roles, systems, data entities, goals) and relationships (information flows, decision dependencies, cause-effect links). The result is a graph of how the business functions.
- **Create Ontologies on the Fly:** They can start without a rigid ontology – letting the LLM suggest categories and connections freely based on context, then iteratively refining them. In early stages, the AI might propose relationships that seem sensible; the graph thinker can validate and adjust. Over time, a more stable ontology can emerge, but it's built bottom-up from actual data rather than imposed top-down. This addresses the limitation of static taxonomies that *“don't reflect the organic and evolutionary nature of reality”* in organizations ³².
- **Visualize the “World Model” of the Company:** With graphs visualized, patterns emerge. For instance, one might see that a particular data source is feeding many critical reports (a hub in the graph), indicating a single point of truth – or failure. Or one might notice two teams talk about the same concept with different terms; the graph can link those as equivalent or related, exposing a hidden dependency. As Dinis Cruz found, the power is not in forcing everyone into one language, but in *“connecting entities between two different graphs”*, mapping relationships across teams without demanding full standardization ³³ ³⁴. In a company knowledge graph, this could mean linking the Sales team's concept of “lead” to the Marketing team's concept of “prospect” if they are analogous – improving mutual understanding while respecting each team's terminology.
- **Simulate and Analyze:** Once a business process or org chart exists as an actual graph structure (nodes might be tasks, roles, systems; edges might be information flows or sequence), graph algorithms can be run. For example, one could run shortest path to see how information travels from frontline customer feedback to the product development team – maybe it's too many hops

(departments) and causing delay. Or identify bottleneck nodes with high centrality (everyone goes through one person or system, which could be a risk). An LLM could assist in natural language: “highlight any processes that lack redundancy” or “what teams have no direct communication path to customer support?” – and then generate answers based on the graph.

- **Continuous Evolution:** Because GenAI can continually ingest new data (new project docs, changes in personnel, updates in software architecture), the knowledge graph can stay up-to-date. Instead of a one-time consulting deliverable (a binder that starts collecting dust immediately), the graph becomes a living resource. Graph thinkers can set up scheduled AI jobs that scan for changes (e.g., new data systems added, new hires and reporting lines, policy changes) and automatically update the relevant part of the graph. This ties to the idea of *self-improving knowledge graphs* ³⁵ ³⁶ – graphs that don’t remain static but evolve as the organization does, aided by AI to handle the scaling of information.

The end result for businesses is a **“digital twin” of their knowledge and processes**, far more faithful and queryable than any org chart or Confluence wiki page. It can be used for training new employees, diagnosing inefficiencies, compliance checks (tracing data lineage for regulations), and strategic decision support (seeing the ripple effects of a change). For graph thinkers, this is a perfect playground: they get to apply their holistic view to real problems, and now they have the tools to implement it largely on their own. This democratizes what used to require big consulting engagements or enterprise software projects.

2. Personal Knowledge Management and Education

On a personal level, individuals can leverage these techniques to build their own **knowledge graphs** and learning tools. Imagine a student or researcher who thinks in graphs – they could use LLMs to turn their notes, references, and hypotheses into a graph of interconnected ideas. There are already efforts to use AI for turning unstructured notes into structured graphs (for instance, extracting who-knows-who from a set of historical letters, or concepts from a textbook chapter and how they relate). With an **ephemeral LLM graph** approach, a learner can:

- Ingest their reading material through an AI, which extracts a graph of key concepts and how they relate.
- Query it: “How does concept X relate to concept Y?” and get a generated explanation with a path through the graph.
- Continuously update the graph with new insights or corrections, essentially having a personal knowledge base that’s both visual and queryable.

This could revolutionize study techniques, allowing one to see the “map of knowledge” of a subject rather than just linear notes. It also closely aligns with how graph thinkers naturally learn – non-linearly and by connecting dots across domains.

3. Creative Brainstorming and Innovation

Graph thinkers often excel in creative brainstorming because they can link ideas from different fields. With GenAI, they can amplify this capability:

- Use an LLM to generate a graph of a problem space, including unrelated domains that might have analogous solutions (a technique known as **analogical mapping**). For instance, if working on a supply chain problem, ask the AI to connect concepts from biology (food webs) or computer networks to spark cross-domain insights.

- Quickly prototype solution models: If they have an idea for a new product or social initiative, they can sketch its components and interactions as a graph, have AI generate code to simulate parts of it, or even a simple UI to test the concept, all without a full development team.

4. Semantic Web and Linked Data for All

The semantic web community has long advocated for linked data and ontologies to make information more machine-understandable. However, it remained a niche activity often requiring expertise in RDF/SPARQL, OWL, etc. Now, graph thinkers can use GenAI to bridge their informal understanding with formal semantic graphs. For example, they could describe a domain and have the AI generate an ontology (in, say, OWL or a JSON-LD context) that they can refine and publish. The AI can also help align their custom ontology with existing ones by finding matches. Dinis Cruz's work hints at this, where the LLM is initially allowed to freely define entities and relationships, and later steps involve refining those with more deterministic ontology inputs ³⁷ ³⁸. This two-step (creative then refining) process can be done by one person with AI assistance, rather than needing a whole committee to agree on standards before any data gets linked.

5. AI Agent Memory and Decision-Making

For those developing AI agents or advanced decision-support systems, having a graph-structured memory is advantageous. An agent that “thinks in graphs” could, for instance, maintain a graph of its observations and use it for reasoning (some projects like **Agentic Graphs** or memory systems like Zep's knowledge graphs are exploring this ³⁹). Graph thinkers could design better AI agents by specifying how information is structured as a graph, and using GenAI to continuously update and prune that graph. This merges human intuition about what concepts are important with the AI's ability to process large volumes of data.

Collaboration: Graph Thinkers and Traditional Developers

Empowering graph thinkers with GenAI and no-code tools does **not** eliminate the need for traditional software developers and engineers. On the contrary, it opens up new modes of collaboration. The roles and workflows will adjust in the following ways:

- **Prototyping vs. Production:** Graph thinkers, with their LLM-driven prototypes, can rapidly create *proof-of-concept* solutions or models. These might function well enough for internal use, one-off analysis, or demos. However, taking an idea to a reliable, scalable production system may require hardening by professional developers. The graph thinker can hand over a working model (and perhaps the intermediate graph artifacts, schema, etc.) to developers who then implement the critical parts in a more robust manner – e.g., optimizing a slow piece of code, moving a time-critical function from Python/AI to a lower-level language, adding proper user authentication and security checks, etc.
- **Identifying the Gaps:** Because graph thinkers can now build much more of the system by themselves, the collaboration with developers becomes more focused. Instead of developers working with a blank slate specification, they're presented with a partially working system and concrete gaps or pain points to address. For example, a graph thinker's LLM-based solution might have a component that calls an API for data and occasionally fails or is too slow. Developers can spot that and say, “We'll build a caching layer for this,” or “We'll implement this part as a microservice.” It's a more efficient use of engineering time on the **non-negotiable requirements** (performance, security, compliance, etc.) that GenAI prototypes often lack ⁴⁰

⁴¹ .

- **Leveraging Developers' Domain Knowledge:** Developers experienced in building graph systems (like those who worked with Neo4j, etc.) bring valuable knowledge on what pitfalls to avoid. If a graph thinker's approach is novel but reinvents something already known (perhaps an indexing strategy, or a way to avoid exponential blow-up in graph search), developers can guide the improvement. The collaboration is almost mentor-like, where the graph thinker still drives the vision, and developers bring the execution expertise. This dynamic is more rewarding for both sides: the graph thinker feels ownership and immediate progress, the developer sees clearly how their specialized skills solve a problem and learn from the domain expert in return.
- **Maintaining Graphs Long-Term:** If the project becomes core to an organization, traditional dev practices (version control, continuous integration, rigorous testing) must be applied. Developers can integrate the outputs of the graph thinker's AI workflow into a pipeline that is monitored and tested. For instance, if an LLM is used regularly to update a knowledge graph, developers might create automated tests on the output (to ensure no hallucinated or incorrect edges are introduced) and set up alerting if something goes wrong. They may even containerize an open-source LLM with a fixed prompt to ensure consistency, rather than always calling a third-party API that might change. In short, they **productionize the AI+graph solution**.
- **Continuous Innovation Loop:** Ideally, organizations will recognize the synergy of pairing graph-thinking individuals with engineering teams. As GenAI takes over routine coding, developers can focus on *frameworks and infrastructure* that amplify what those individuals can do. It becomes a cycle: graph thinker conceives and prototypes → developers fortify and scale it → resulting tool allows graph thinker (and others) to go after even bigger problems. This loop can drive an innovation culture where ideas don't stagnate and practical solutions emerge faster.

It's worth noting that developers themselves might become graph thinkers over time, once exposed to these tools. Conversely, graph thinkers might pick up more technical skills since the AI removes the most tedious parts of coding, leaving the more interesting bits accessible. The boundary is blurring – which is in itself a hallmark of the *democratization of development*.

Crucially, the fear that “AI will replace developers” is reframed here: **AI empowers a new class of creators**, and developers evolve to higher-level problem solvers and educators within the process. Just as past waves of automation elevated the level of abstraction at which humans operate (assembly code to high-level languages to frameworks), GenAI and no-code elevate us to a more conceptual, design-oriented plane of work. Graph thinkers exemplify that by operating directly at the knowledge model level, and developers ensure those models are realized efficiently and safely.

Benefits and Future Outlook

The confluence of graph thinking and generative AI capabilities is still in its early days, but the trajectory is clear and promising. Here we outline the key benefits of embracing this approach, and what the future might hold:

1. Unleashing Creativity and Innovation: By removing the traditional bottlenecks, individuals who were once sidelined in the creation process can now actively build and experiment. Organizations that encourage their “idea people” or subject-matter experts to use these AI-assisted graph tools could see a surge in innovative solutions. Problems that were previously thought intractable or too costly to address (due to analysis paralysis or lack of engineering resources) might be solved with a small AI-augmented team rapidly prototyping and iterating on a graph model of the problem.

2. Better Decision Making through Explainability: One criticism of end-to-end AI solutions has been the *black box* nature of results. By incorporating knowledge graphs and intermediate representations, we gain explainability. Each result or recommendation can be traced through the nodes and edges (or the chain of LLM prompt steps) that led to it. In the MyFeeds example, instead of the AI magically picking 5 relevant news articles for a persona, the process involves extracting entities, mapping them to the persona's interests, and then selecting articles with the highest overlap – with each step documented in JSON and graphs ⁴² ³⁸. This way, *provenance* is built into the system: every statement can be connected back to a source ⁴³. For businesses, government, or science, such transparency is invaluable. It builds trust in AI-assisted decisions and allows audits and improvements.

3. Adaptive Knowledge Systems: In the future, knowledge graphs created with these techniques will not be static artifacts; they will be living systems that *learn and adapt*. With human-in-the-loop feedback (graph thinkers or any user correcting or enhancing the graph), and with GenAI continuously processing new inputs, these graphs will update themselves. Dinis Cruz envisions graphs that “*evolve and get better the more they're used*” ⁴⁴ ⁴⁵. For instance, if many people query a knowledge graph and frequently correct it or navigate it in certain ways, the AI could suggest reorganizing the graph for clarity, or merging nodes that users often consider equivalent. This turns knowledge management into a dynamic conversation rather than a periodic manual curation task.

4. Bridging Silos and Perspectives: Graphs naturally merge data from different silos. GenAI can take data that was never meant to work together – say, a SQL database of sales figures and an Excel sheet of marketing campaigns – and link them via common entities (like product names or regions). This is often where breakthroughs occur, by seeing a *connection between two things that were previously disconnected*. On a human level, graph thinking augmented by AI can also bridge perspectives. As described in the “ontologies of ontologies” concept ⁴⁶ ⁴⁷, we can maintain diversity (different teams or individuals have their own way of structuring knowledge) while still achieving unity by mapping between those structures. AI can help automate those mappings and maintain them even as each sub-ontology evolves. The result is a federated but integrated knowledge network – a very powerful notion for large enterprises, research communities, or even global challenges that require interdisciplinary collaboration.

5. New Roles and Skills: We might see the rise of roles like “AI-facilitated Knowledge Modeler” or “Graph Systems Architect” which are essentially what graph thinkers become when formally empowered. These individuals will be adept at prompt engineering for knowledge extraction, using tools like MGraph-DB to refine data, and guiding the overall knowledge strategy of an organization. It's a hybrid of business analyst, data scientist, and ontologist – but enabled by AI to be far more productive than those roles in the past. Educational programs and training might start focusing on **graph literacy** and prompt design, recognizing that this combination is a critical skillset in the modern workforce.

Looking ahead, several trends are likely:

- **Integration of Vector and Symbolic:** LLMs excel at understanding unstructured data (text, images via captions, etc.) and connecting concepts (in a “vector space” sense), while graphs excel at precise symbolic representation and logical reasoning. We will see more systems that seamlessly integrate vector-based AI and symbolic graphs. A query might first use a vector search (to find relevant pieces of info) and then populate a graph and reason over it, all orchestrated by an AI agent. Graph thinkers will play a big role in designing these hybrid workflows, because they naturally think about how to combine different methods in a network of transformations.

- **Standardization and Interchange:** As more knowledge graphs are built by individuals across domains, there will be efforts to allow them to **interconnect**. Imagine if personal or organizational knowledge graphs could be selectively merged or queried across, with privacy controls. There might be protocols for AI-agents to exchange graph fragments. Open-source projects could arise to host common ontologies for things like supply chain, healthcare, cybersecurity (an area Dinis comes from). GenAI can assist in mapping any custom graph to these standards, lowering the barrier to contributing. This could fulfill some of the original semantic web vision, but in a more organic way driven by AI and individual creators rather than committees alone.
- **Continuous Improvement of LLMs for Graph Tasks:** AI research is likely to produce LLM variants or finetuned models specialized in graph reasoning. Already, projects are looking at how to reduce hallucinations by giving the model a structured “memory” or by splitting tasks into smaller steps (which is essentially what the multi-phase approach does ⁴⁸ ⁴⁹). We may get models that can take larger contexts (thus bigger graphs) or that have built-in understanding of graph algorithms. This will make the ephemeral graph database approach even more powerful, as the AI will be more reliable and capable in performing complex graph queries accurately.
- **Visualization and UX Advances:** The output of all this work often needs to be consumed by end users who may not be tech-savvy. We expect more intuitive interfaces where users can chat with a graph (ask questions and see the subgraph as part of the answer), or explore a knowledge graph visually with AI guidance (like a GPS for knowledge: “you are here, these are nearby concepts, you might want to see this connection...”). Graph thinkers, aided by AI, will likely craft novel ways to present interconnected information, moving beyond static node-link diagrams to richer, multi-dimensional views – possibly integrating timelines, geospatial info, or interactive storytelling on top of graphs.

Conclusion

The emergence of generative AI as a co-pilot in development has finally opened the door for those who **think in graphs** to directly bring their visions to life. No longer constrained by the need for extensive coding or by the rigidity of legacy tools, these individuals can now **leverage LLMs as both collaborators and engines** for graph-based innovation. We have outlined how the synergy of graph thinking and GenAI – supported by memory-first graph databases like MGraph-DB and serverless architecture using simple storage – creates a powerful platform for building semantic knowledge graphs on-demand, iteratively, and at low cost.

The impact of this shift is multifaceted. We will see faster and more transparent problem-solving in businesses as hidden connections are unveiled through AI-generated graphs. We will empower a broader community to engage in knowledge modeling – turning knowledge once trapped in experts’ minds or static documents into living graphs that can be queried, visualized, and improved continuously. In doing so, organizations and individuals become more **adaptive and insightful**, as they can map and navigate the complexities of their domains with unprecedented clarity.

Crucially, this is a story of **democratization**. The ability to create sophisticated graph-based solutions is no longer the sole province of specialized software companies or PhD data scientists. A passionate professional with the right mindset and AI tools can achieve in weeks what might have taken a large team months or years in the past. The learning curve for technology is flattening; GenAI is the great equalizer, turning natural language into working systems and freeing creativity from the shackles of syntax and setup.

However, this does not make human expertise any less important – in fact, it elevates the importance of creative and systems thinking. The role of the human shifts to providing vision, context, critical evaluation, and ethical guidance, while the AI handles the grunt work of implementation and data crunching. The most successful outcomes will arise from **close collaboration between graph-thinking domain experts and savvy developers**, combining conceptual brilliance with technical robustness.

In the coming years, we anticipate a flourishing ecosystem of **open-source tools, shared ontologies, and community-driven knowledge graphs**, many initiated by individuals empowered by GenAI. Challenges remain – from ensuring data quality and AI accuracy to managing security in AI-driven workflows – but these are surmountable with thoughtful design and collaboration (as we discussed, developers will play a key role in this). The trajectory is clear: knowledge, once siloed and static, is becoming **connected and alive**.

As we conclude, we circle back to the initial vision: a person who sees connections everywhere can now materialize those connections into a tangible graph, ask "what if?" and get answers, refine their ideas and see them grow, all at the speed of conversation. The walls between imagination and realization are crumbling. In this new paradigm, *everything can truly be a graph* – and anyone with the passion to explore it can harness that graph for insight and impact.

Sources:

- Cruz, Dinis. *Building Semantic Knowledge Graphs with LLMs: Inside MyFeeds.ai's Multi-Phase Architecture*. LinkedIn, Mar 24, 2025 ⁵⁰ ⁴³ .
- Cruz, Dinis. *Introducing: MGraph-AI - A Memory-First Graph Database for GenAI and Serverless Apps*. LinkedIn, Jan 11, 2025 ¹⁶ ¹⁷ .
- Cruz, Dinis. *This is really cool and powerful! The MGraph-DB timeline is now automatically created and published to S3...* LinkedIn post, Feb 20, 2025 ²⁴ ²⁷ .
- Cruz, Dinis. *Beyond Static Ontologies: How GenAI Powers Self-Improving Knowledge Graphs*. LinkedIn, Mar 31, 2025 ³⁵ ³⁴ .
- Microsoft Research. *GraphRAG: Unlocking LLM discovery on narrative private data*. Feb 13, 2024 ¹¹ ¹² .
- Flow Ninja Blog. *How Generative AI Will Change Low-code/No-code Development*. 2023 ² .
- (Additional references within text from linked sources as cited in context above.)

¹ ⁵ ⁶ ⁹ ¹⁰ ²⁰ ²¹ ³¹ ³⁷ ³⁸ ⁴² ⁴³ ⁴⁸ ⁴⁹ ⁵⁰ Building Semantic Knowledge Graphs with LLMs: Inside MyFeeds.ai's Multi-Phase Architecture
<https://www.linkedin.com/pulse/building-semantic-knowledge-graphs-llms-inside-myfeedsais-dinis-cruz-jub5e>

² ³ ⁴ ⁸ How Generative AI Will Change Low-code/No-code Development
<https://www.flow.ninja/blog/how-generative-ai-will-change-low-code-no-code-development>

⁷ Exploring Low/No-Code Platforms, GenAI, Copilots ... - DevOps.com
<https://devops.com/exploring-low-no-code-platforms-genai-copilots-and-code-generators/>

11 12 GraphRAG: Unlocking LLM discovery on narrative private data - Microsoft Research

<https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/>

13 14 15 16 17 18 19 28 29 30 Introducing: MGraph-AI - A Memory-First Graph Database for GenAI and Serverless Apps

<https://www.linkedin.com/pulse/introducing-mgraph-ai-memory-first-graph-database-genai-dinis-cruz-wxmde>

22 23 24 25 26 27 This is really cool and powerful! The MGraph-DB timeline is now... | Dinis Cruz

https://www.linkedin.com/posts/diniscruz_this-is-really-cool-and-powerful-the-activity-7298377848216580096-Oq3U

32 33 34 35 36 44 45 46 47 Beyond Static Ontologies: How GenAI Powers Self-Improving Knowledge Graphs

https://www.linkedin.com/pulse/beyond-static-ontologies-how-genai-powers-knowledge-graphs-dinis-cruz-ombpe?trk=public_post

39 Agentic Knowledge Graphs: A Living, Learning AI Architecture

<https://www.linkedin.com/pulse/agentic-knowledge-graphs-living-learning-ai-rick-gillespie-3g4ue>

40 41 PDF: Proposal for Neo4j Collaboration with Dinis Cruz | Dinis Cruz

https://www.linkedin.com/posts/diniscruz_proposal-for-neo4j-collaboration-with-dinis-activity-7335623659497189376-7L2t